

3.11. Dadas las siguientes transacciones

$T_1 = l_1(A); r_1(A); l_1(B); w_1(B); u_1(A); u_1(B)$

$T_2 = l_2(C); r_2(C); l_2(A); w_2(A); u_2(C); u_2(A)$

$T_3 = l_3(B); r_3(B); l_3(C); w_3(C); u_3(B); u_3(C)$

$T_4 = l_4(D); r_4(D); l_4(A); w_4(A); u_4(D); u_4(A)$

Se pide:

- Transformar las mismas a un modelo de locks ternario
- Realizar un entrelazado tal que se produzca un deadlock y todas las transacciones queden esperando
- Realizar el grafo *wait-for*
- Describir que ocurre en el esquema anterior si se utiliza prevención de *deadlocks* con
 - Un esquema Wait-Die
 - Un esquema Wound-Wait

a) $T_1 = r_1(A); r_1(A); w_1(B); w_1(B); u_1(A); u_1(B)$

$T_2 = r_2(C); r_2(C); w_2(A); w_2(A);$
 $u_2(C); u_2(A)$

$T_3 = r_3(B); r_3(B); w_3(C); w_3(C); u_3(B);$
 $u_3(C); u_3(B)$

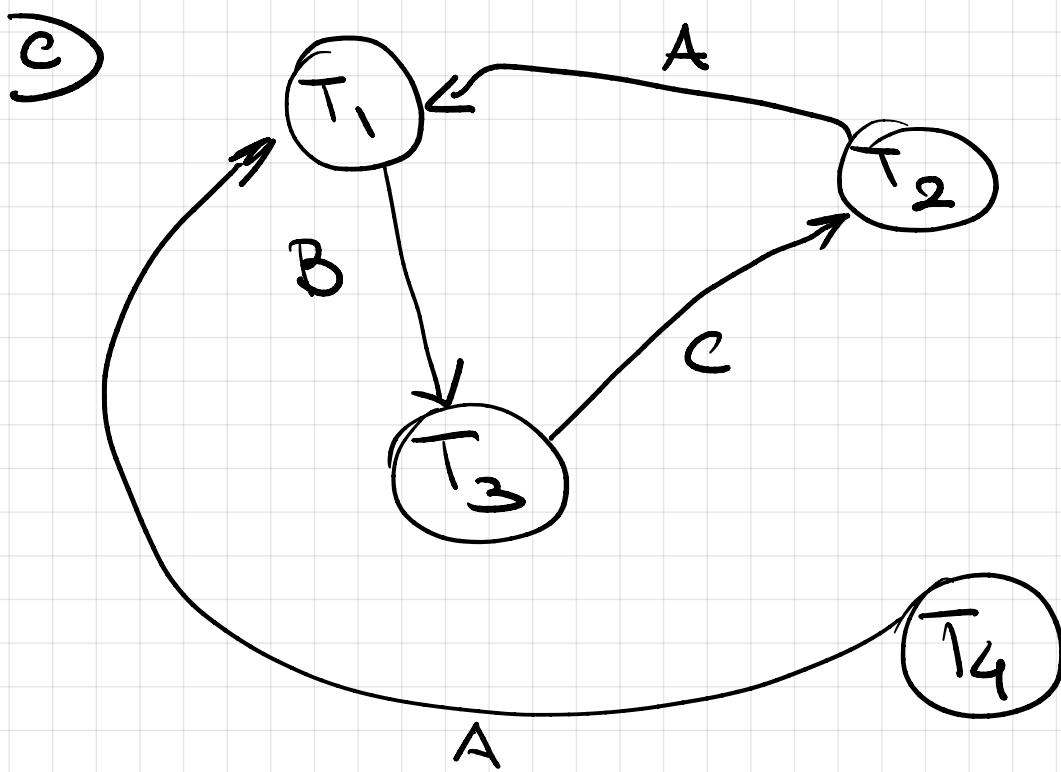
$T_4 = r_4(D); r_4(D); w_4(A); w_4(A);$
 $u_4(D); u_4(A)$

b)

T_1	T_2	T_3	T_4
$rl_1(A)$ $r_1(A)$			
	$rl_2(C)$ $r_2(C)$		
		$rl_3(B)$ $r_3(B)$	
			$rl_4(D)$ $r_4(D)$
$wl_1(B)$			
	$wl_2(A)$		
		$wl_2(C)$	
			$wl_4(A)$

Esta historia no
es legal, ¿está bien?

¿La legalidad de la historia, no
protege de deadlocks?



Hay un ciclo: $T_3 \rightarrow T_2 \rightarrow T_1$

Deadlock

d) ① $T_1 = 0$	$T_2 = 1$	$T_3 = 2$	$T_4 = 3$
wait die $\ell_1(A)$ $r_1(A)$			
	$\ell_2(C)$ $r_2(C)$		
		$\ell_3(B)$ $r_3(B)$	
			$\ell_4(D)$ $r_4(D)$
$wl_1(B)$ se pone en espera			
	$wl_2(A)$ Abort T_2		
		$wl_3(C)$ T_2 fue abortada $\ell_2(C)$ no existe $w_3(C)$	
se soluciona el problema del dead lock T_2 y T_3 es ejecutada luego con mismo TS			$wl_4(A)$ $TS(T_4) > TS(T_1)$ Abort T_4

Wound Wait $T_1 = 0$	$T_2 = 1$	$T_3 = 2$	$T_4 = 3$
① $rl_1(A)$ $r_1(A)$			
	$rl_2(C)$ $r_2(C)$		
		$rl_3(B)$ $r_3(B)$	
			$rl_4(D)$ $r_4(D)$
$wl_1(B)$ Aborto a T_3 que tiene el lock y es más nova. $w_1(B)$		X	
	$wl_2(A)$ el lock lo tiene T_1 se pone en espera		
T_1 Puede seguir ejecutando y el deadlock desaparece.			$wl_4(A)$ Espera a T_1